

CẢI TIẾN THUẬT TOÁN XẾP HẠNG ĐỒ THỊ ỨNG DỤNG VÀO THIẾT KẾ MẠCH QUY MÔ LỚN VLSI

1st Đỗ Đăng Khoa

Khoa Công Nghệ Thông Tin 1
Học viện Công nghệ Bưu chính Viễn thông
Hà Nội, Việt Nam
khoadd.b21cn068@stu.ptit.edu.vn

2nd Nguyễn Đình Sơn

Khoa Công Nghệ Thông Tin 1
Học viện Công nghệ Bưu chính Viễn thông
Hà Nội, Việt Nam
sonnd.b21dccn652@stu.ptit.edu.vn

3rd Đỗ Ngọc Đăng

Khoa Công Nghệ Thông Tin 1
Học viện Công nghệ Bưu chính Viễn thông
Hà Nội, Việt Nam
dangdn.b22bn206@stu.ptit.edu.vn

4th Đỗ Tiến Dũng

Khoa Công Nghệ Thông Tin 1
Học viện Công nghệ Bưu chính Viễn thông
Hà Nội, Việt Nam
dungdt@ptit.edu.vn

Tóm tắt—Trong thiết kế vật lý mạch tích hợp quy mô rất lớn (VLSI), bước đặt chỗ (placement) quyết định trực tiếp các chỉ số công suất, hiệu năng và diện tích. Ở các thiết kế hiện đại, các macro chiếm phần lớn diện tích và chiều dài dây, nên thứ tự xử lý macro có ảnh hưởng mạnh đến chất lượng sản xuất. Về công cụ đặt chỗ truyền thống ta có DREAMPlace, còn MaskPlace là công cụ đặt macro tuần tự sử dụng mạng nơ-ron tích chập và học tăng cường, do đó đặc biệt nhạy với thứ tự macro. Bài báo đề xuất một luồng hai giai đoạn là xếp hạng đồ thị toàn cục và tích hợp vào DREAMPlace để xác định thành phần có vai trò quan trọng sau đó xếp hạng đồ thị tập trung vào các macro cố định bằng các thuật toán làm đầu vào cho DREAMPlace và MaskPlace. Kết quả trên bộ chuẩn ISPD2005 cho thấy thứ tự macro do thuật toán PageRank sinh ra giúp giảm tới khoảng 14,67% chiều dài dây bán chu vi (HPWL) và 2–6% thời gian chạy so với cấu hình mặc định, với chi phí xếp hạng tăng thêm không đáng kể.

Từ khóa— Đặt chỗ VLSI, Xếp hạng nút đồ thị, DREAMPlace, MaskPlace, Macro placement, Giảm thời gian chạy, Tối ưu HPWL.

I. GIỚI THIỆU

A. Thiết kế VLSI và bài toán đặt chỗ

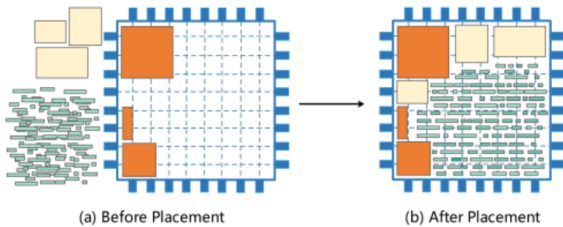
Thiết kế các mạch tích hợp quy mô rất lớn liên quan đến việc tích hợp hàng nghìn, triệu hoặc

thậm chí hàng tỷ bóng bán dẫn vào một chip duy nhất [1]. Thiết kế VLSI hoạt động ở quy mô rất chi tiết, thường dao động từ 0,1 đến 0,01 micromet, và bao gồm các quy trình phức tạp của việc thiết kế và thử nghiệm IC [1], [2], [3]. Trong quy trình thiết kế vật lý, hai giai đoạn quan trọng được xác định là đặt chỗ cho các thành phần lớn như RAM và ROM, và đặt chỗ các ô tiêu chuẩn, bao gồm các cổng logic như NOR, NAND [4], [5], [6]. Sau bước đặt chỗ, các công cụ **định tuyến (route)** sẽ thực hiện kết nối các net; toàn bộ quy trình này thường được gọi chung là placement & route (đặt chỗ và định tuyến).

Trong các thiết kế vi mạch hiện đại, thường có hai hướng tiếp cận đặt chỗ là đặt chỗ phân tích và đặt chỗ dựa trên học máy. Ở bất kỳ cách tiếp cận nào, thành phần macro thường chiếm 50–90% diện tích đặt và đóng góp 60–80% tổng HPWL cuối cùng. Điều này khiến vị trí và thứ tự xử lý macro trở thành yếu tố then chốt đối với chất lượng đặt chỗ.

Quá trình đặt chỗ phân tích (analytical placement) thường được chia làm ba giai đoạn là **đặt chỗ toàn cục (global placement)**, hợp pháp hóa và đặt chỗ chi tiết. Trong đó, việc đặt chỗ toàn cục cho phép đặt vị trí gần đúng của các ô, chấp nhận chồng chéo một phần [1], [7], [8]. Bước hợp pháp

hóa sẽ giải quyết việc chồng chéo và các thiết kế vi phạm quy tắc. Cuối cùng, bước đặt chỗ chi tiết thực hiện các tinh chỉnh để nâng cao chất lượng của bo mạch. Theo quy trình hoạt động như trên, bước đặt chỗ toàn cục giữ vai trò quan trọng nhất, tiêu tốn phần lớn thời gian của toàn bộ quy trình, khiến nó trở thành mục tiêu được tối ưu hóa trong nhiều công trình [3], [9]. Bên cạnh đó, quá trình đặt chỗ dựa trên học máy được sử dụng để đặt vị trí mỗi ô linh kiện trên bề mặt chip 2D và hướng đến tối ưu hóa bằng cách tối đa hóa các chỉ số cụ thể với cơ chế thưởng phạt của học tăng cường, từ đó tạo trực tiếp ra các bố cục hợp lệ mà không cần qua bước hợp pháp hóa hoặc đặt chỗ chi tiết như trong phương pháp phân tích [10], [11]. Quy trình đặt chỗ dựa trên học máy mạng đồ thị chuyển đổi các liên kết giữa các thành phần trên bo mạch thành các đồ thị với các nút là các thành phần trên bo mạch và đường mạch nối giữa chúng là các cạnh. Mục tiêu là xác định tọa độ của các thành phần phải gắn vào bo mạch sao cho đồng thời (i) tối thiểu hóa các hàm mục tiêu của học máy và (ii) tuân thủ đầy đủ các quy định thiết kế [12]. Minh họa cho quá trình đặt chỗ được thể hiện trong Hình 1.



Hình 1: Quá trình đặt chỗ.

Quá trình xếp đặt thường diễn ra trong một vùng hình chữ nhật, có thể chia thành nhiều ô nhỏ để tổ chức dễ dàng hơn. Vùng xếp đặt này được minh họa bằng các hình chữ nhật màu xanh dương. Trong vùng này, các hình chữ nhật màu cam đại diện cho macro cố định, màu trắng sữa là macro có thể di chuyển và màu xanh lá là các cell tiêu chuẩn có thể di chuyển [1]. Các hàm mục tiêu thường liên quan đến chiều dài dây, mật độ, tắc nghẽn và các ràng buộc không chồng chéo, nhằm tối ưu hóa khả năng định tuyến cũng như các chỉ số PPA toàn hệ thống.

B. Các hướng tiếp cận và chiến lược hai giai đoạn

Nhiều hướng tiếp cận khác nhau đã được đề xuất để tăng cường khả năng tối ưu đặt chỗ. Các phương pháp dựa trên học tăng cường và mạng nơ-ron đồ thị cho thấy tiềm năng lớn trong việc thiết kế đầu cuối các bố trí macro và standard cell [6], [13], [10]. DREAMPlace đại diện cho lớp công cụ phân tích dựa trên gradient descent, trong đó toàn bộ macro và standard cell được tối ưu đồng thời; cách làm này giúp tận dụng hiệu quả phần cứng GPU nhưng không áp đặt thứ tự ưu tiên rõ ràng cho từng macro. Các công trình như RePlace, ePlace, DeepPlace hay DeepPR cũng theo hướng này, tập trung vào cải tiến hàm chi phí và quá trình hội tụ, ít khai thác thông tin về thứ hạng của các thành phần.

Trong một hướng khác, MaskPlace sử dụng mạng nơ-ron tích chập để đặt các macro tuần tự trên bo mạch. Mỗi macro được gán vị trí tại một bước trong chuỗi, nên *thứ tự macro* trở thành đầu vào quan trọng: macro được xử lý sớm có nhiều tự do hơn, còn macro ở cuối dễ rơi vào vùng kẹt hoặc biên. Trong phiên bản gốc, thứ tự này được học bằng học tăng cường qua rất nhiều lần huấn luyện, khó giải thích và tốn kém chi phí [6], [11], [12], [14].

Trên bối cảnh đó, nghiên cứu trong bài báo này được thiết kế thành **hai giai đoạn liên tiếp**:

- **Giai đoạn 1 – Xếp hạng toàn cục trong DREAMPlace:** netlist được chuyển sang dạng đồ thị, sau đó nhiều kịch bản xếp hạng khác nhau được áp dụng trên toàn mạng: ranking by nets, xếp hạng toàn bộ node, chỉ các node cố định (fixed only), chỉ các node di động (movable only), với các thuật toán như Eigenvector Centrality (EGC) và PageRank. Các thứ tự sinh ra được đưa vào DREAMPlace dưới dạng các chiến lược sắp xếp dữ liệu đầu vào và ưu tiên cập nhật.

- **Giai đoạn 2 – Xếp hạng macro-centric với MaskPlace + DREAMPlace:** dựa trên các kết quả giai đoạn 1, trong đó các kịch bản xếp hạng trên nhóm phần tử cố định (fixed-only) cho hiệu quả tốt nhất, giai đoạn 2 thu hẹp đối tượng xuống *các macro cố định* có độ kết nối cao. Các thuật toán PageRank, EGC và Degree Centrality được áp dụng trên đồ thị macro để sinh điểm xếp hạng, sau đó dùng thứ tự này trong MaskPlace và trọng số trong DREAMPlace, qua đó đánh giá định

lượng chất lượng HPWL macro và thời gian chạy.

Cách tổ chức này cho phép vừa khảo sát rộng hiệu ứng của xếp hạng đồ thị (giai đoạn 1), vừa đào sâu vào bài toán macro-centric nơi macro đóng vai trò trung tâm (giai đoạn 2), tất cả đều nằm trong một luồng nghiên cứu thống nhất.

C. Đóng góp của bài báo

Trên cơ sở hai giai đoạn nêu trên, nội dung bài báo có thể tóm tắt qua ba đóng góp chính:

- **Khung xếp hạng đồ thị hai tầng cho placement:** giai đoạn thứ nhất xếp hạng toàn cục các phần tử netlist và tích hợp vào DREAMPlace để giảm thời gian chạy, cho thấy hiệu quả của ranking và gợi ý vai trò then chốt của nhóm phần tử cố định. Giai đoạn thứ hai tập trung vào *macro-centric ranking*: áp dụng PageRank, EGC và Degree chỉ trên các macro cố định có độ kết nối cao, xem đây là tập nút quan trọng nhất của đồ thị.

- **Tích hợp xếp hạng macro với MaskPlace và DREAMPlace trong cùng một luồng:** lớp xếp hạng macro được chèn như một tầng tiền xử lý cho MaskPlace, cung cấp thứ tự macro rõ ràng thay vì để quá trình học tăng cường tự học, đồng thời được dùng để điều chỉnh trọng số trong hàm chi phí của DREAMPlace. Vai trò khác biệt của hai công cụ được làm rõ: MaskPlace là môi trường thể hiện trực tiếp hiệu ứng của thứ tự macro, trong khi DREAMPlace đánh giá ảnh hưởng lên quá trình tối ưu toàn cục và thời gian chạy.

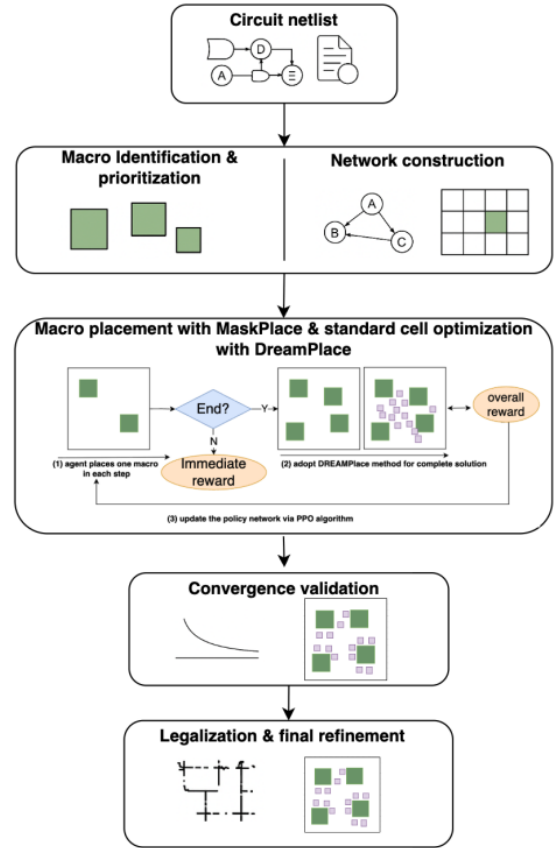
- **Đánh giá định lượng và xác định thuật toán xếp hạng hiệu quả nhất:** các kịch bản xếp hạng khác nhau được kiểm chứng trên bộ dữ liệu ISPD2005 và các thiết kế công nghiệp, với các chỉ số gồm thời gian chạy tổng thể, thời gian đặt chỗ phi tuyến và HPWL chỉ tính macro. Kết quả chỉ ra rằng thứ tự macro cố định do PageRank sinh ra cho mức giảm HPWL macro trung bình 14,67% (tối đa 25,32%) và giảm 2–6% thời gian chạy so với cấu hình mặc định, vượt trội so với EGC và Degree. Điều này khẳng định PageRank là thuật toán xếp hạng phù hợp nhất cho bài toán macro-centric trong bối cảnh kết hợp MaskPlace và DREAMPlace.

II. PHƯƠNG PHÁP

A. Kiến trúc tổng thể

Hình 2 mô tả luồng tổng thể của phương pháp đề xuất, kết hợp lớp xếp hạng đồ thị macro với

MaskPlace và DREAMPlace trong một pipeline thống nhất, từ netlist đầu vào đến bố trí cuối cùng đã hợp pháp hóa.



Hình 2: Luồng tổng thể: xếp hạng macro, đặt macro bằng MaskPlace và tối ưu standard cell bằng DREAMPlace.

Luồng làm việc gồm các bước chính sau:

- **Circuit netlist:** bắt đầu từ netlist mức mạch, gồm macro, standard cell và các net kết nối giữa chúng. Netlist được phân tích cú pháp để trích xuất thông tin loại phần tử, kích thước và các ràng buộc cố định/di động.

- **Macro identification & prioritization và network construction:** các macro được nhận diện, phân loại thành *macro cố định* và *macro di động*. Từ netlist xây dựng một đồ thị, trong đó mỗi macro là một nút, các cạnh biểu diễn các net chung. Trên đồ thị này, các thuật toán PageRank, Eigenvector Centrality và Degree Centrality được áp dụng để tính điểm quan trọng của từng macro; kết quả tạo thành danh sách ưu tiên macro. Song song với đó, một lưới không gian (grid) và các đặc trưng trạng thái được xây dựng cho môi trường MaskPlace và cho hàm chi phí của DREAMPlace.

- **Macro placement với MaskPlace & tối ưu standard cell với DREAMPlace:** trong mỗi bước của MaskPlace, quá trình học tăng cường chọn một macro (theo thứ tự ưu tiên do lớp xếp hạng cung cấp) và gán vị trí trên lưới. Sau mỗi hành động, một *immediate reward* được tính toán dựa trên các đặc trưng proxy như HPWL ước lượng, mức chồng lấn và mật độ. Khi tất cả macro đã được đặt (*End? = Yes*), DREAMPlace được gọi để thực hiện global placement cho standard cell và tinh chỉnh toàn bộ bố trí, sử dụng điểm xếp hạng macro như trọng số trong hàm chi phí. Từ bố trí hoàn chỉnh này, một *overall reward* được tính cho toàn episode; mạng chính sách của agent MaskPlace được cập nhật bằng thuật toán PPO dựa trên cả *immediate reward* và *overall reward*.

- **Convergence validation:** quá trình huấn luyện và tối ưu được lặp lại cho đến khi các tiêu chí hội tụ về HPWL, mật độ và thời gian chạy đạt ngưỡng đặt trước. Giai đoạn này tương ứng với hộp “Convergence validation” trong Hình 2, nơi các đường cong mục tiêu dần ổn định.

- **Legalization & final refinement:** sau khi hội tụ, bố trí kết quả được đưa qua bước hợp pháp hóa nhằm loại bỏ chồng lấn và đảm bảo tuân thủ hoàn toàn các quy tắc thiết kế. Cuối cùng, một bước tinh chỉnh cục bộ (ví dụ NTUPlace DP [15]) được áp dụng để tối ưu thêm HPWL và cải thiện tính khả định tuyến của thiết kế.

Cách tổ chức này phản ánh rõ vai trò của từng thành phần: xếp hạng đồ thị macro cung cấp thứ tự và trọng số, MaskPlace thể hiện trực tiếp tác động của thứ tự macro trong quá trình đặt chỗ tuần tự, còn DREAMPlace đảm nhiệm tối ưu và hoàn chỉnh bố trí trước khi hợp pháp hóa.

B. Vai trò của học máy và học tăng cường trong luồng đề xuất

Trong luồng đề xuất, học máy xuất hiện ở hai lớp với vai trò khác nhau. DREAMPlace là một bộ đặt chỗ phân tích được hiện thực trên hạ tầng học sâu: hàm chi phí (HPWL, mật độ) được cài đặt dưới dạng các toán tử khả vi trên GPU và tối ưu bằng gradient descent. DREAMPlace không huấn luyện mô hình theo nghĩa cổ điển (không có pha training trên tập dữ liệu lớn với nhãn), mà chủ yếu tận dụng thư viện deep learning để giải một bài toán tối ưu hoá số liên tục.

Ngược lại, MaskPlace sử dụng mạng nơ-ron tích chập kết hợp với học tăng cường (như PPO)

để đặt macro tuần tự trên bo mạch. Ở mỗi bước, mô hình quan sát trạng thái lưới, chọn vị trí cho macro tiếp theo, nhận phần thưởng tức thời, và sau khi DREAMPlace hoàn thiện bố trí, nhận thêm phần thưởng tổng thể để cập nhật chính sách. Nhờ đó, MaskPlace thay thế heuristic đặt macro truyền thống bằng một chính sách được học cho nhiều thiết kế khác nhau.

Lớp xếp hạng đồ thị trong bài báo chỉ đóng vai trò *tiền xử lý nhẹ* cho hai công cụ này. Trong MaskPlace, thứ tự macro đầu vào được sinh từ các thuật toán xếp hạng (PageRank, EGC, Degree) thay vì cố định hoặc để mô hình tự khám phá toàn bộ, giúp thu hẹp không gian tìm kiếm. Trong DREAMPlace, điểm xếp hạng được ánh xạ thành trọng số trong hàm chi phí để các macro quan trọng hội tụ nhanh hơn, trong khi lỗi gradient descent vẫn giữ nguyên. Chi phí bổ sung chủ yếu nằm ở bước xây dựng đồ thị và tính các chỉ số trung tâm, có độ phức tạp xấp xỉ tuyến tính theo số nút/cạnh và nhỏ so với thời gian chạy tổng thể. Đối lại, các kết quả ở Bảng III và Bảng IV cho thấy HPWL macro được cải thiện trung bình 14,67% và thời gian chạy giảm 2–6% so với cấu hình mặc định, nên việc đánh đổi giữa chi phí AI bổ sung và lợi ích hiệu năng là có lợi.

C. Lựa chọn các thuật toán xếp hạng đồ thị

Các thuật toán xếp hạng được sử dụng trong bài có bản chất khác nhau và phản ánh những góc nhìn riêng về tầm quan trọng của nút trên đồ thị.

Degree Centrality đo trực tiếp số lượng cạnh nối với mỗi nút. Trong đồ thị macro, bậc cao nghĩa là macro đó kết nối với nhiều macro hoặc standard cell khác. Thuật toán đơn giản, dễ tính nhưng không phân biệt được giữa các kết nối “mạnh/yếu” và không phản ánh chất lượng của nút hàng xóm.

Eigenvector Centrality (EGC) mở rộng Degree theo nguyên lý “một nút quan trọng nếu được nối với các nút cũng quan trọng”. Điểm số là một véc-tơ riêng của ma trận kề, nhấn mạnh các vùng có cấu trúc nổi trội. Tuy nhiên, EGC nhạy với phân bố nhóm của đồ thị; trong các đồ thị có cụm dày hoặc cấu trúc không đều, điểm số dễ tập trung vào một vài vùng và làm giảm độ ổn định trong bài toán đặt chỗ.

PageRank có thể xem là một biến thể ổn định hơn của EGC, dựa trên mô hình “người lướt web ngẫu nhiên”: ở mỗi bước, người dùng đi theo một

cạnh với xác suất $1 - \alpha$, hoặc “nhảy” đến một nút bất kỳ với xác suất α (hệ số damping). Cơ chế damping giúp tránh việc điểm số bị dồn vào các chu trình hoặc cụm khép kín, đồng thời phản ánh xác suất một nút được “ghé thăm” trong dài hạn. Trên đồ thị macro netlist với nhiều cụm kết nối và các macro lớn đóng vai trò các cụm quan trọng, PageRank có xu hướng gán điểm ổn định cho các macro vừa có nhiều kết nối, vừa nằm trên các đường truyền quan trọng.

Nhờ sự cân bằng giữa thông tin bậc (degree), chất lượng các nút xung quanh và cơ chế damping, PageRank tỏ ra phù hợp hơn cho bài toán xếp hạng macro trong luồng MaskPlace + DREAMPlace. Điều này được phản ánh trong các kết quả thực nghiệm (Bảng III, Bảng IV), nơi PageRank cho mức cải thiện HPWL macro và thời gian chạy ổn định hơn so với Degree và Eigenvector Centrality.

D. Lớp xếp hạng đồ thị toàn cục cho DREAMPlace (Giai đoạn 1)

Tầng xếp hạng toàn cục (giai đoạn 1) được thiết kế nhằm trả lời hai câu hỏi: (i) liệu xếp hạng đồ thị có thực sự giúp giảm thời gian chạy so với cấu hình không xếp hạng và (ii) nhóm phần tử nào (fixed, movable, nets, all nodes) là nguồn lợi chính.

Các bước thực hiện:

- 1) **Xây dựng đồ thị toàn cục:** từ netlist, một đồ thị ở mức cell/net được xây dựng; các node biểu diễn cell (bao gồm macro và standard cell), các cạnh biểu diễn kết nối giữa chúng.
- 2) **Xây dựng các kịch bản ranking:** nhiều kịch bản xếp hạng khác nhau được định nghĩa, ví dụ:
 - **Ranking by nets:** xếp hạng dựa trên trọng số net.
 - **Edge_all_node:** xếp hạng toàn bộ node theo các chỉ số độ trung tâm.
 - **EGC_Fixed_Only:** chỉ xếp hạng trên tập node cố định.
 - **PageRank_Fixed_Only:** chỉ xếp hạng trên tập node cố định.
 - **EGC_Movable_Only:** chỉ xếp hạng trên tập node di động.
 - **PageRank_Movable_Only:** chỉ xếp hạng trên tập node di động.

- 3) **Tích hợp vào DREAMPlace:** thứ tự xếp hạng được dùng để sắp xếp dữ liệu đầu vào cũng như ưu tiên trong các bước cập nhật nội bộ của DREAMPlace. Cấu trúc lõi của công cụ không bị thay đổi; lớp ranking hoạt động như một tầng tiền xử lý.
- 4) **Đánh giá:** thời gian chạy tổng thể và thời gian đặt chỗ phi tuyến được đo trên bộ dữ liệu ISPD2005. Các kết quả cho thấy các kịch bản ranking dựa trên fixed-only cho hiệu quả tốt nhất, trong khi movable-only kém hiệu quả hơn.

Giai đoạn này khẳng định rằng ranking có ích và gợi ý mạnh rằng nhóm phần tử cố định, tương ứng với các macro lớn, nên trở thành đối tượng ưu tiên trong các bước cải tiến tiếp theo.

E. Lớp xếp hạng macro-centric cho MaskPlace và DREAMPlace (Giai đoạn 2)

Trong giai đoạn 2, đối tượng được xếp hạng được thu hẹp xuống các macro cố định có độ kết nối cao. Đây là bước tiến trực tiếp từ các gợi ý của tầng toàn cục.

- 1) **Xây dựng đồ thị mức macro:** từ netlist, một đồ thị mới được trích xuất, trong đó mỗi macro cố định tương ứng với một nút. Hai macro được nối với nhau nếu cùng xuất hiện trong một net. Độ lớn macro và số lượng net kết nối được lưu làm thuộc tính nút/cạnh.
- 2) **Tính điểm xếp hạng:** các thuật toán PageRank, Eigenvector Centrality (EGC) và Degree Centrality được áp dụng trên đồ thị này để thu được véc-tơ điểm số cho các macro. PageRank phản ánh mức độ quan trọng toàn cục, EGC phản ánh tầm quan trọng trong việc chi phối cấu trúc của đồ thị, còn Degree nhấn mạnh số lượng kết nối trực tiếp.
- 3) **Sử dụng điểm xếp hạng trong MaskPlace và DREAMPlace:**
 - *Trong MaskPlace:* danh sách macro cố định được sắp xếp giảm dần theo điểm xếp hạng và dùng trực tiếp làm thứ tự đặt macro. Vì MaskPlace đặt macro tuần tự, các macro quan trọng (điểm cao) sẽ được đặt sớm, có nhiều không gian và tự do hơn, do đó ảnh hưởng mạnh đến chất lượng HPWL macro. Điều này cho phép đánh giá trực tiếp chất lượng của từng thuật toán xếp hạng (PageRank/EGC/Degree) trong bối cảnh macro

placement.

- Trong *DREAMPlace*: sau khi macro được MaskPlace đặt, điểm xếp hạng vẫn được sử dụng để điều chỉnh trọng số trong hàm chi phí của *DREAMPlace*. Các macro có điểm cao được nhân với hệ số trọng số lớn hơn trong thành phần HPWL và mật độ, khiến gradient của chúng có độ lớn lớn hơn và hội tụ nhanh hơn về vị trí ổn định. Standard cell được tối ưu như cấu hình mặc định, nhưng chịu ảnh hưởng gián tiếp từ vị trí macro đã được ưu tiên.

Sự khác biệt giữa *DREAMPlace* và MaskPlace làm rõ ý nghĩa của macro-centric ranking: *DREAMPlace* không khai thác thứ tự macro một cách trực tiếp vì tối ưu tất cả phần tử cùng lúc, trong khi MaskPlace lại phụ thuộc mạnh vào thứ tự đặt macro. Nhờ đó, việc chứng minh PageRank là thuật toán xếp hạng tốt nhất cho macro trong luồng MaskPlace + *DREAMPlace* trở nên có ý nghĩa rõ ràng hơn, thay vì chỉ là hiệu ứng phụ của một bộ tối ưu gradient.

F. Bộ dữ liệu và kịch bản thí nghiệm

Các phương pháp được đánh giá trên bộ dữ liệu chuẩn ISPD2005 – một tập hợp các thử nghiệm với độ phức tạp tăng dần. Bộ dữ liệu bao gồm bốn mạch VLSI (Adaptecl–4), với các thông số đặc trưng là số lượng cell, số lượng net, số cell có thể di chuyển (movable) và số cell cố định (fixed), như trong Bảng I.

Bảng I: Chi tiết về bộ dữ liệu kiểm thử

Design	Cell	Nets	Movs	Fixed
<i>adaptecl</i>	211K	221142	210904	543
<i>adaptecl2</i>	255K	266009	254457	566
<i>adaptecl3</i>	452K	466758	450927	723
<i>adaptecl4</i>	496K	515951	494716	1329

Các kịch bản thí nghiệm chính gồm:

- **Xếp hạng toàn cục với *DREAMPlace* (giai đoạn 1):** đánh giá tác động của nhiều kịch bản ranking (all nodes, nets, fixed-only, movable-only) lên thời gian chạy và quá trình tối ưu phi tuyến; tập trung vào việc xác nhận hiệu quả chung của ranking và vai trò của nhóm fixed.
- **Xếp hạng macro cố định với MaskPlace + *DREAMPlace* (giai đoạn 2):** chỉ xếp hạng các macro cố định bằng PageRank/EGC/Degree, sử dụng thứ tự này trong MaskPlace và trọng số trong

DREAMPlace, sau đó đánh giá HPWL macro và thời gian chạy so với cấu hình mặc định.

III. KẾT QUẢ THÍ NGHIỆM

A. Thiết lập thực nghiệm và cấu hình đánh giá

Các thí nghiệm được thực hiện trên máy chủ GPU với CPU Intel Xeon, GPU NVIDIA Tesla T4 (16 GB), bộ nhớ hệ thống 64 GB, sử dụng hệ điều hành Linux, CUDA và PyTorch cho các phần xếp hạng và MaskPlace, C++/CUDA cho *DREAMPlace*. Bộ dữ liệu đánh giá là ISPD2005 với bốn mạch Adaptec1–4, như trình bày trong Bảng I. Mỗi cấu hình được chạy lặp lại nhiều lần để giảm nhiễu do dao động hệ thống, các giá trị trong bảng được báo cáo dưới dạng trung bình.

Để gắn chặt giữa phần đề xuất và phần thực nghiệm, các thí nghiệm được tổ chức thành hai nhóm chính tương ứng với hai giai đoạn của luồng:

1) *Giai đoạn 1: Xếp hạng toàn cục trong *DREAMPlace*:* Mục tiêu của giai đoạn này là đánh giá (i) liệu việc bổ sung lớp xếp hạng đồ thị trước *DREAMPlace* có giúp giảm thời gian chạy so với cấu hình mặc định hay không và (ii) nhóm phần tử nào (fixed, movable, nets, all nodes) mang lại lợi ích lớn nhất. Các cấu hình được sử dụng trong Bảng II và Hình 3 được định nghĩa như sau:

- **Default without sort:** *DREAMPlace* gốc, không áp dụng bất kỳ chiến lược xếp hạng hay sắp xếp lại thứ tự phần tử nào. Đây là đường cơ sở để so sánh.

- **Ranking by nets:** các net được xếp hạng theo trọng số (ví dụ số lượng kết nối, độ dài ước lượng) và thông tin này được dùng để sắp xếp thứ tự xử lý trong *DREAMPlace*. Cấu hình này kiểm tra giả thuyết rằng ưu tiên các kết nối quan trọng có thể cải thiện hội tụ.

- **Edge all node:** toàn bộ node (macro và standard cell) được gán điểm độ trung tâm dựa trên cấu trúc đồ thị và sau đó được sắp xếp lại thứ tự. Cấu hình này đánh giá tác động của xếp hạng toàn cục lên thời gian chạy.

- **Pagerank all:** tương tự Edge all node nhưng sử dụng PageRank để tính điểm thay vì độ trung tâm phổ. Mục đích là so sánh các chỉ số độ trung tâm khác nhau khi áp dụng trên toàn bộ phần tử.

- **Edge fixed only / Pagerank fixed only:** chỉ các phần tử cố định (fixed), vốn trong bộ benchmark chủ yếu là các macro lớn, được đưa vào bài toán xếp hạng. Các điểm số này được dùng để ưu tiên xử lý nhóm fixed trong DREAMPlace. Hai cấu hình này kiểm chứng giả thuyết nêu trong phần đề xuất rằng nhóm phần tử cố định là nhóm quan trọng nhất của đồ thị và việc ưu tiên chúng sẽ mang lại cải thiện đáng kể về thời gian chạy.

- **Edge movable only / Pagerank movable only:** chỉ các phần tử di động (movable) được xếp hạng và ưu tiên. Các cấu hình này đóng vai trò đối chứng với nhóm fixed, giúp trả lời câu hỏi liệu việc tập trung vào movable thay vì fixed có mang lại lợi ích tương tự hay không.

Trong tất cả các cấu hình trên, thuật toán lõi của DREAMPlace (hàm chi phí, chiến lược gradient descent) được giữ nguyên; lớp xếp hạng chỉ thay đổi thứ tự xử lý phần tử và một số thứ tự dữ liệu nội bộ. Các chỉ số đánh giá chính trong giai đoạn này là **thời gian chạy tổng thể** và **thời gian đặt chỗ phi tuyến**, nhằm đo trực tiếp tác động của xếp hạng lên hiệu năng công cụ. HPWL tổng thể vẫn được kiểm soát để đảm bảo các thay đổi về runtime không đánh đổi bằng việc suy giảm chất lượng lời giải.

2) *Giai đoạn 2: Xếp hạng macro cố định với MaskPlace + DREAMPlace:* Giai đoạn thứ hai tập trung kiểm chứng phần đề xuất *macro-centric* và so sánh các thuật toán xếp hạng khác nhau trên tập macro cố định, sử dụng bộ đôi MaskPlace + DREAMPlace như trình bày trong Hình 2. Các cấu hình trong Bảng III, Bảng IV, Hình 4 và Hình 5 được định nghĩa như sau:

- **Default:** MaskPlace được sử dụng với thứ tự macro mặc định (ví dụ theo thứ tự xuất hiện trong netlist hoặc thứ tự do cấu hình gốc của MaskPlace sinh ra), sau đó DREAMPlace thực hiện global placement cho standard cell. Đây là đường cơ sở để so sánh trong giai đoạn macro-centric.

- **Pagerank:** các macro cố định được trích xuất, xây dựng thành một đồ thị mức macro, sau đó được xếp hạng bằng PageRank. Danh sách macro đầu vào cho MaskPlace được sắp xếp giảm dần theo điểm PageRank và điểm này đồng thời được ánh xạ thành trọng số trong hàm chi phí của DREAMPlace. Cấu hình này trực tiếp kiểm thử giả thuyết rằng PageRank là thuật toán xếp hạng phù hợp nhất cho bài toán macro-centric.

- **Eigenvector:** tương tự cấu hình Pagerank nhưng sử dụng Eigenvector Centrality để xếp hạng các macro. Cấu hình này là đối chứng cho PageRank nhằm đánh giá tác động của một chỉ số độ trung tâm phổ khác.

- **Degree:** các macro được xếp hạng theo bậc (degree), tức số lượng kết nối trực tiếp trong đồ thị macro. Đây là một baseline đơn giản để so sánh với hai chỉ số nâng cao hơn là PageRank và Eigenvector Centrality.

Trong giai đoạn này, MaskPlace luôn đảm nhiệm việc đặt macro tuần tự, còn DREAMPlace chịu trách nhiệm tối ưu liên tục và đặt standard cell; lớp xếp hạng chỉ can thiệp vào *thứ tự macro đầu vào cho MaskPlace* và *trọng số macro trong hàm chi phí của DREAMPlace*. Các chỉ số đánh giá chính gồm:

- **HPWL macro:** chỉ tính chiều dài dây nửa chu vi của các macro, phản ánh trực tiếp chất lượng của chiến lược xếp hạng trong bối cảnh macro placement.

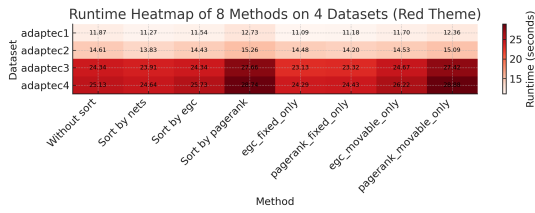
- **Thời gian chạy tổng thể:** bao gồm cả thời gian MaskPlace và DREAMPlace, dùng để đánh giá tác động của các cấu hình xếp hạng lên hiệu năng thực thi.

Việc tách rõ hai giai đoạn và các cấu hình tương ứng cho phép gắn trực tiếp kết quả thực nghiệm với các đóng góp đã nêu: giai đoạn 1 kiểm chứng hiệu quả tổng quát của xếp hạng đồ thị và vai trò của nhóm phần tử cố định trong DREAMPlace, trong khi giai đoạn 2 xác định thuật toán xếp hạng macro tối ưu (PageRank) trong luồng MaskPlace + DREAMPlace và định lượng mức cải thiện HPWL cũng như thời gian chạy.

B. Kết quả giai đoạn 1: xếp hạng toàn cục trong DREAMPlace

Giai đoạn 1 tập trung vào tăng xếp hạng toàn cục. Khung thực nghiệm sử dụng Python và PyTorch cho phần xếp hạng, C++/CUDA trên GPU NVIDIA Tesla T4 cho DREAMPlace. Cấu hình *Default without sort* tương ứng với cách sử dụng gốc của DREAMPlace không có lớp xếp hạng đồ thị.

Bảng II và Hình 3 cho thấy các kịch bản “Ranking by nets”, “Edge all node”, EGC_Fixed_Only và PageRank_Fixed_Only đều giúp giảm thời gian chạy so với cấu hình không xếp hạng, với mức giảm từ 1% đến khoảng 6%. Đặc biệt, EGC_Fixed_Only và PageRank_Fixed_Only cho



Hình 3: Thời gian chạy tổng thể (giây) cho từng benchmark trong DREAMPlace với các thuật toán xếp hạng khác nhau (giai đoạn 1).

giảm thời gian chạy trung bình khoảng 2,8–3,2%, trong khi các kịch bản chỉ xếp hạng movable cho kết quả kém ổn định hơn, thậm chí tăng runtime ở một số benchmark.

Bảng II: So sánh thời gian chạy tổng thể (giây) trong DREAMPlace (giai đoạn 1)

Dataset	Default without sort	Ranking by nets	Edge all node
adaptec1	11,87	11,27 -5,05%	11,54 -2,78%
adaptec2	14,61	13,77 -5,75%	14,38 -1,57%
adaptec3	24,43	23,44 -4,06%	24,12 -1,27%
adaptec4	25,13	24,64 -1,95%	24,37 -3,02%

Dataset	Pagerank all	Edge fixed only	Pagerank fixed only
adaptec1	12,73 +7,25%	11,09 -6,57%	11,18 -5,81%
adaptec2	15,26 +4,45%	14,13 -3,29%	14,42 -1,30%
adaptec3	27,66 +13,23%	24,53 +0,41%	24,23 -0,82%
adaptec4	27,94 +11,18%	24,29 -3,34%	24,18 -3,78%

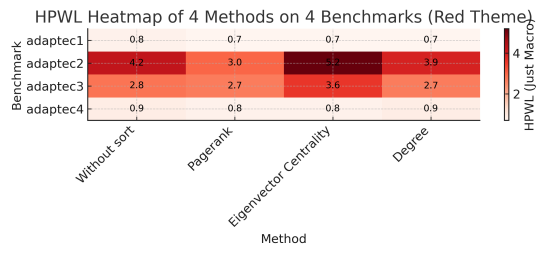
Dataset	Edge movable only	Pagerank movable only
adaptec1	11,70 -1,43%	12,36 +4,13%
adaptec2	14,53 -0,55%	15,09 +3,29%
adaptec3	24,27 -0,65%	25,87 +5,89%
adaptec4	26,52 +5,53%	27,82 +5,35%

Kết quả này khẳng định rằng xếp hạng đồ thị là hữu ích cho DREAMPlace và đồng thời chỉ ra nhóm phần tử cố định là nguồn lợi chính, từ đó cung cấp cơ sở thực nghiệm cho giai đoạn macro-centric tiếp theo.

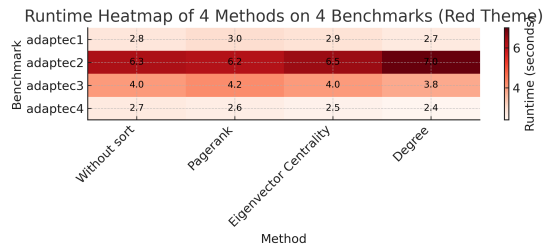
C. Kết quả giai đoạn 2: xếp hạng macro cố định với MaskPlace + DREAMPlace

Giai đoạn 2 tập trung vào đánh giá chất lượng của các thuật toán xếp hạng trên tập macro cố định, sử dụng MaskPlace như môi trường thể hiện trực tiếp tác động của thứ tự macro, và DREAMPlace như tăng tinh chỉnh và đánh giá thêm về runtime.

Bảng III và Hình 4 cho thấy PageRank vượt trội trong việc giảm HPWL macro, đạt mức cải thiện trung bình đáng kể là -14,67% và đỉnh điểm giảm tới -25,32% trên Adaptec2 so với cấu hình Default (MaskPlace sử dụng thứ tự mặc định). Thành công này bắt nguồn từ khả năng xác định chính xác các nút kết nối quan trọng trong đồ thị macro. Ngược lại, EGC làm tăng HPWL trung



Hình 4: HPWL (chỉ macro) cho các benchmark từ Adaptec1 đến Adaptec4 với các thuật toán xếp hạng khác nhau (Default, PageRank, Eigenvector centrality và Degree).



Hình 5: Thời gian chạy (giây) của các thuật toán xếp hạng khác nhau trên nhiều bộ dữ liệu đánh giá (MaskPlace + DREAMPlace).

bình +7,64%, chủ yếu do suy giảm mạnh trên Adaptec2 và Adaptec3, nơi đặc tính cấu trúc của EGC dẫn đến vị trí đặt chưa tối ưu. Thuật toán Degree cho kết quả cải thiện vừa phải nhưng ổn định (-6,77% trung bình), cân bằng giữa sự đơn giản và hiệu quả, dù thiếu đi độ tinh vi về nhận thức mạng như PageRank.

Bảng IV và Hình 5 cho thấy PageRank cũng cải thiện runtime một cách ổn định (-2,64% trung bình), đạt đỉnh -7,17% trên Adaptec3. Degree đạt mức giảm runtime trung bình cao hơn (-3,42%) nhưng không ổn định, runtime tăng trên Adaptec2. EGC đạt kết quả kém nhất với mức giảm runtime không đáng kể (-0,71%) và thậm chí tăng runtime trên Adaptec1 và Adaptec2. Các kết quả này chứng minh rằng việc ưu tiên macro cố định theo PageRank vừa nâng cao chất lượng lời giải (HPWL) vừa giữ được hiệu quả tính toán trong quy trình đặt chỗ các thành phần trên bo mạch.

Một điểm quan trọng là: vì DREAMPlace tối ưu tất cả phần tử cùng lúc, tác động của thứ tự macro trong DREAMPlace chỉ xuất hiện gián tiếp, thông qua việc ranking thay đổi trình tự xử lý dữ liệu nội bộ. Trong khi đó, MaskPlace phụ thuộc trực tiếp vào thứ tự macro do đặt tuần tự. Do vậy, việc chứng minh PageRank là thuật toán xếp

Bảng III: HPWL (chỉ tính macro) trên các bộ dữ liệu đánh giá khác nhau (MaskPlace + DREAMPlace, giai đoạn 2).

Dataset	Default	Pagerank	Eigenvector	Degree
Adaptec1	733,264	596,826 -18,61%	623,808 -14,93%	629,333 -14,17%
Adaptec2	4.110,780	3.069,908 -25,32%	5.196,827 +26,42%	3.802,726 -7,49%
Adaptec3	2.932,513	2.801,201 -4,48%	3.562,688 +21,49%	2.845,711 -2,96%
Adaptec4	829,228	744,145 -10,26%	784,712 -5,37%	858,754 +3,56%

Bảng IV: Thời gian chạy (giây) của các thuật toán xếp hạng khác nhau (MaskPlace + DREAMPlace, giai đoạn 2).

Dataset	Default	Pagerank	Eigenvector	Degree
Adaptec1	2,851.9	2,836.9 -0,53%	2,918.4 +2,33%	2,640.0 -7,43%
Adaptec2	6,358.8	6,285.7 -1,15%	6,525.3 +2,62%	7,060.0 +11,03%
Adaptec3	4,248.4	3,944.0 -7,17%	4,119.0 -3,05%	3,970.0 -6,55%
Adaptec4	2,935.6	2,854.1 -2,78%	2,855.6 -2,73%	2,650.0 -9,73%

hạng tốt nhất cho macro trên nền MaskPlace mang ý nghĩa rõ ràng: thứ tự macro cố định dựa trên PageRank thực sự cải thiện chất lượng bố trí, chứ không chỉ là hiệu ứng phụ của một bộ tối ưu gradient.

IV. KẾT LUẬN

Bài báo này xây dựng một khung xếp hạng đồ thị hai tầng cho bài toán đặt chỗ VLSI trong một luồng thống nhất. Giai đoạn xếp hạng toàn cục chỉ ra rằng việc xếp hạng các phần tử netlist trước khi chạy DREAMPlace giúp giảm 1–6% thời gian chạy, đồng thời gợi ý rằng nhóm phần tử cố định (macro lớn, nhiều kết nối) mang lại hiệu quả tốt nhất khi được ưu tiên. Trên cơ sở đó, giai đoạn macro-centric tập trung riêng vào các macro cố định, sử dụng PageRank, EGC và Degree để xếp hạng, sau đó khai thác thứ tự macro trong MaskPlace và trọng số trong DREAMPlace.

Các kết quả thực nghiệm trên bộ chuẩn ISPD2005 cho thấy thứ tự macro cố định do PageRank sinh ra giúp giảm trung bình 14,67% HPWL macro (tối đa 25,32%) và giảm 2–6% thời gian chạy so với cấu hình mặc định của MaskPlace và DREAMPlace, vượt trội so với EGC và Degree. Điều này khẳng định rằng (i) xếp hạng đồ thị là một bước tiền xử lý hữu ích, (ii) macro cố định là đối tượng nên được ưu tiên trong các chiến lược ranking và (iii) trong số các thuật toán xếp hạng được khảo sát, PageRank là lựa chọn phù hợp nhất cho bài toán macro-centric trên nền MaskPlace + DREAMPlace. Hướng phát triển tiếp theo là kết hợp sâu hơn học tăng cường

và mạng nơ-ron đồ thị để vừa học được biểu diễn, vừa học được hàm xếp hạng phù hợp cho các hệ thống VLSI đa lớp, đa mục tiêu.

Tài liệu tham khảo

- [1] Y. Qiu, Y. Xing, X. Zheng, P. Gao, S. Cai, and X. Xiong, “Progress of placement optimization for accelerating vlsi physical design,” *Electronics*, vol. 12, no. 2, 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/2/337>
- [2] K. Itoh, “Vlsi memory chip design,” 2013. [Online]. Available: <https://books.google.com.vn/books?id=ojnvCAAQBAJ>
- [3] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, “Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement,” 2019. [Online]. Available: <https://doi.org/10.1145/3316781.3317803>
- [4] G.-J. Nam, C. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, “The ispd2005 placement contest and benchmark suite,” pp. 216–220, 04 2005.
- [5] Z. W. Jiang, H. C. Chen, T. C. Chen, and Y. W. Chang, “Challenges and solutions in modern vlsi placement,” pp. 1–5, Apr. 2007.
- [6] Y. Lai, Y. Mu, and P. Luo, “Maskplace: Fast chip placement via reinforced visual representation learning,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 019–24 030, 2022.
- [7] K. Shahookar and P. Mazumder, “Vlsi cell placement techniques,” *ACM Computing Surveys (CSUR)*, vol. 23, no. 2, pp. 143–220, 1991.
- [8] J. Lu, H. Zhuang, P. Chen, H. Chang, C. C. Chang, Y. C. Wong, L. Sha, D. Huang, Y. Luo, C. C. Teng *et al.*, “eplace-ms: Electrostatics-based placement for mixed-size circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 5, pp. 685–698, 2015.
- [9] C. K. Cheng, A. B. Kahng, I. Kang, and L. Wang, “Replace: Advancing solution quality and routability validation in global placement,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 12, pp. 1717–1730, 2019.
- [10] A. Agnesina, S. Pentapati, and S. K. Lim, “A general framework for vlsi tool parameter optimization with deep reinforcement learning,” in *Proceedings of the NeurIPS 2020 Workshop on Machine Learning for Systems*, Dec. 2020, pp. 6–12.
- [11] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y. J. Lee, E. Johnson, O. Pathak,

- A. Nazi *et al.*, “A graph placement methodology for fast chip design,” *Nature*, vol. 594, pp. 207–212, 2021.
- [12] V. Lee, C. Deng, L. Elzeiny, P. Abbeel, and J. Wawrzynek, “Chip placement with diffusion,” arXiv preprint arXiv:2407.12282, 2024.
- [13] D. Vashisht, H. Rampal, H. Liao, Y. Lu, D. Shanbhag, E. Fallon, and L. B. Kara, “Placement in integrated circuits using cyclic reinforcement learning and simulated annealing,” arXiv preprint arXiv:2011.07577, 2020.
- [14] Y. Chen, Z. Wen, Y. Liang, and Y. Lin, “Stronger mixed-size placement backbone considering second-order information,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [15] T. C. Chen, T. C. Hsu, Z. W. Jiang, and Y. W. Chang, “Ntuplace: A ratio partitioning based placement algorithm for large-scale mixed-size designs,” in *Proceedings of the 2005 International Symposium on Physical Design*, Apr. 2005, pp. 236–238.



Đỗ Đăng Khoa Sinh viên nghiên cứu tại Học viện Công nghệ Bưu chính Viễn thông khóa 2021, chuyên ngành Hệ thống thông tin.



Nguyễn Đình Sơn Sinh viên nghiên cứu tại Học viện Công nghệ Bưu chính Viễn thông khóa 2021, chuyên ngành Hệ thống thông tin.



Đỗ Tiến Dũng nhận bằng tiến sĩ ngành Công nghệ y sinh tại Đại học Hokkaido, Nhật Bản năm 2022. Hiện tại, đang làm nghiên cứu và giảng dạy tại Khoa Công nghệ Thông tin 1, Học viện Công nghệ Bưu chính Viễn thông. Hướng nghiên cứu hiện tại là: học máy, xử lý ngôn ngữ tự nhiên, thị giác máy tính, tin sinh.



Đỗ Ngọc Đăng Sinh viên nghiên cứu tại Học viện Công nghệ Bưu chính Viễn thông khóa 2022, ngành Công nghệ Thông Tin.